

Contents

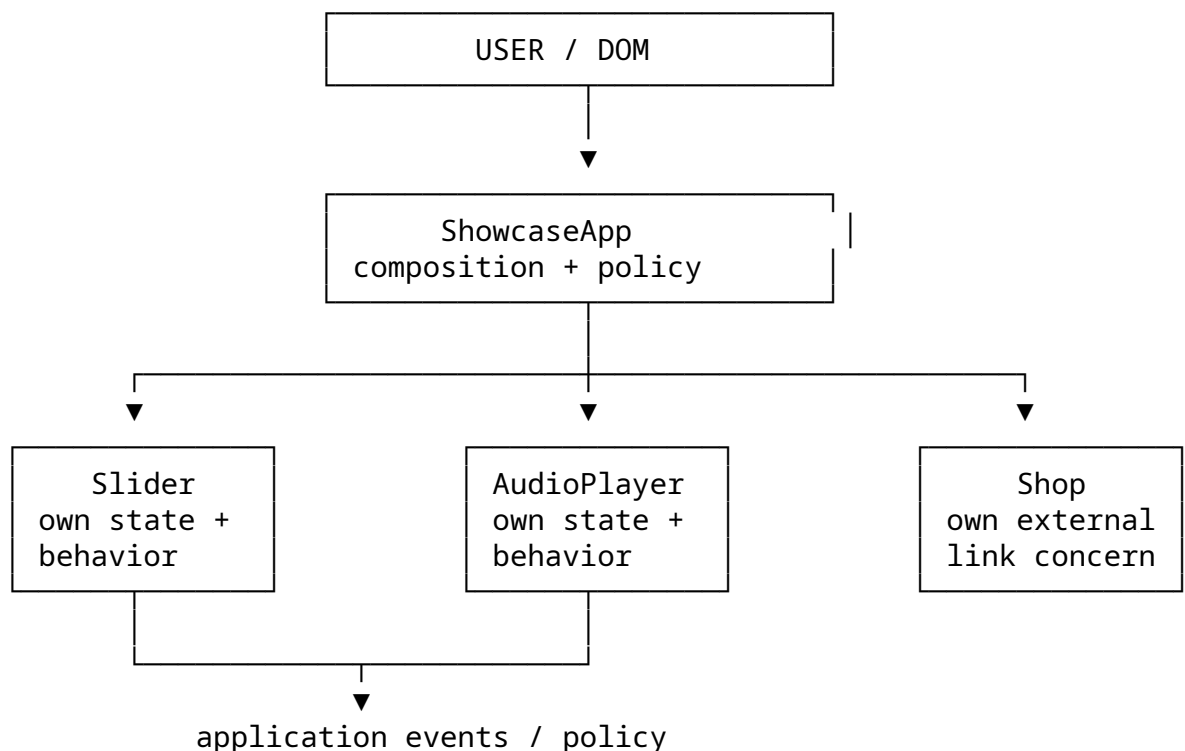
Architecture Overview	2
1. Reading the System as One Idea	2
2. Design Philosophy	2
3. Architectural Layers	3
4. System at a Glance	3
5. Main Architectural Principles	4
5.1 Component encapsulation	4
5.2 Separation of concerns	5
5.3 Configuration over duplication	5
5.4 Fail-fast contracts	6
6. Application Composition	6
7. Components Do Not Directly Own Each Other	7
8. Slider Architecture	7
9. Local State Machines	8
10. Slider FSM	9
11. Autoscroll FSM	9
12. Audio FSM	9
13. Input Pipeline	10
14. Keyboard Pipeline	11
15. Managers	11
16. EventManager	12
16.1 Declarative static maps	12
16.2 Dynamic subscriptions	12
17. KeyboardManager and ButtonManager	13
18. Button	13
19. Other Infrastructure	14
19.1 Timer	14
19.2 DOMValidator	14
19.3 AudioDeckView	14
20. Architectural Pattern Map	14
21. Why There Is No Global State Pattern	15
22. Pattern Interaction	16
23. Architectural Philosophy	17
24. Final Architecture	18
24.1 Summary	18

Architecture Overview

1. Reading the System as One Idea

At first glance the project looks like an interactive slider with an audio player. That description is functionally correct, but architecturally incomplete. The interesting part of the project is what happens when several different interaction domains have to coexist without turning the application into one large object.

The architecture therefore starts from ownership rather than from features. The Slider owns movement. AudioPlayer owns playback. Shop owns its external-link concern. Reusable mechanics such as event subscription, keyboard dispatch, button dispatch and timing are extracted into managers. ShowcaseApp sits above them and coordinates the consequences that cross component boundaries.



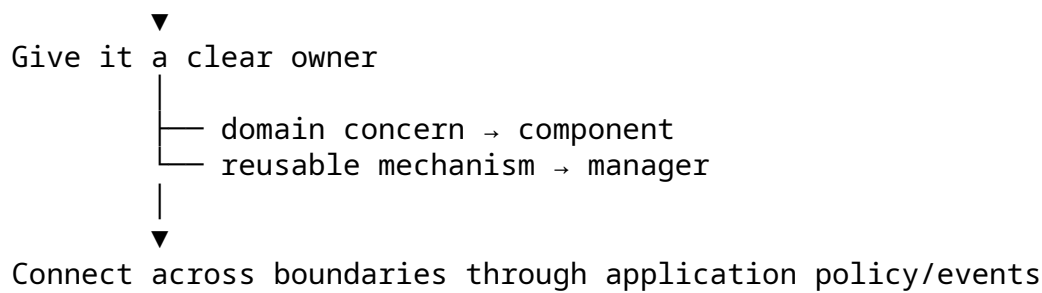
This perspective explains why the project has more structure than a typical slider. Each additional layer exists because another architectural responsibility appeared and deserved an explicit home.

2. Design Philosophy

The central philosophy is **complex but controllable** rather than **minimal but tightly coupled**. This is an educational architecture exercise, so the project deliberately explores what happens when the UI is decomposed into cooperating objects without relying on a framework.

The aim is not abstraction for its own sake. The useful abstraction is the one that gives a concept a clear owner, removes duplication of mechanism, or creates a reusable contract. That is why the system uses managers, local FSMs and a pipeline, while deliberately avoiding a forced global State layer or generic wrapper around every constant.

A real concern appears
|



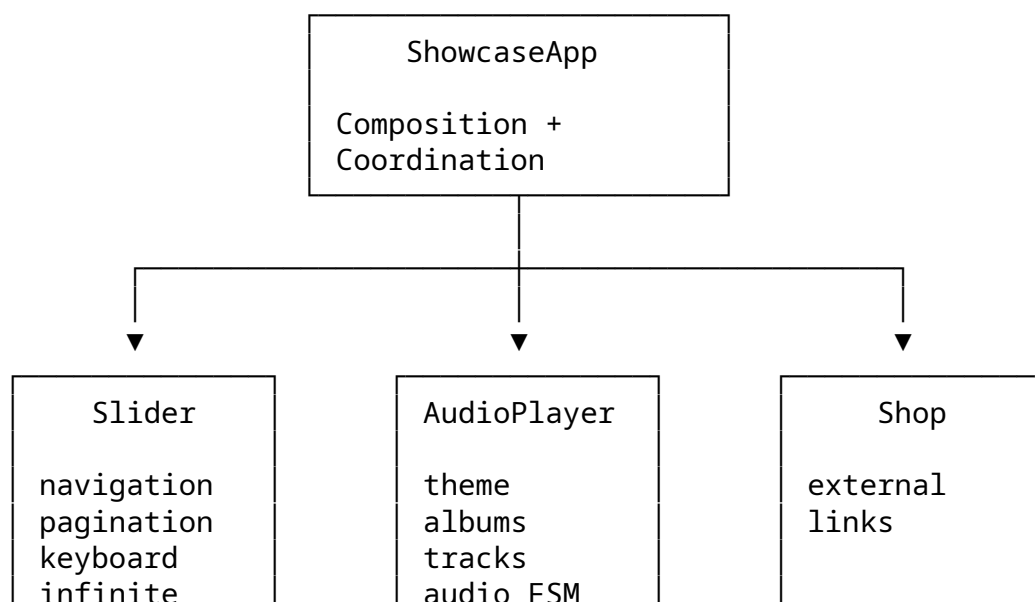
3. Architectural Layers

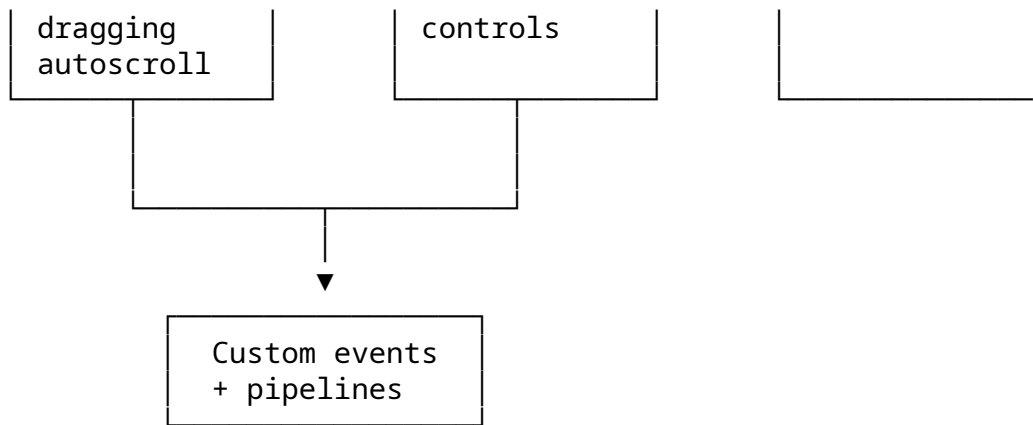
The whole application can be read as five cooperating layers:

1. Application orchestration ShowcaseApp
2. Domain components Slider / AudioPlayer / AudioDeckView / Shop
3. Interaction infrastructure EventManager / KeyboardManager / ButtonManager Timer
4. Contract / UI support DOMValidator / Button
5. Internal mechanisms inheritance / polymorphism / FSM / _pipe

The rest of this document explains why this decomposition is useful and how the parts co-operate at a high level.

4. System at a Glance





The key rule is simple:

Component owns its own domain logic
 +
 Managers own reusable interaction mechanisms
 +
 ShowcaseApp coordinates cross-component behavior
 +
 Events and pipelines connect the parts

5. Main Architectural Principles

Component encapsulation

Each major component owns its own state and behavior.

Slider

- navigation
- pagination
- dragging
- autoscroll
- slider/autoscroll state

AudioPlayer

- playback
- theme
- album/track selection
- audio state

Shop

- external link state

ShowcaseApp coordinates these components but does not reach into their internal implementation.

Separation of concerns

Responsibilities are deliberately split between:

Application

└─ ShowcaseApp

Components

├─ Slider
├─ AudioPlayer
├─ AudioDeckView
└─ Shop

Reusable infrastructure

├─ EventManager
├─ KeyboardManager
├─ ButtonManager
└─ Timer

Validation / UI abstractions

├─ DOMValidator
└─ Button

Utilities

└─ helpers

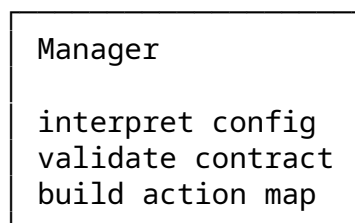
This is why the code is larger than a minimal slider implementation: the project is exploring architecture, not merely UI mechanics.

Configuration over duplication

Interaction behavior is described through configuration tables where practical.

Conceptually:

configuration

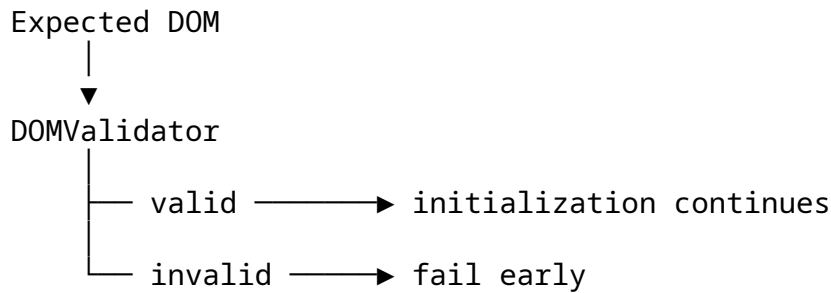


Component

This is particularly visible in keyboard, click and event handling.

Fail-fast contracts

The project deliberately prefers explicit failure over defensive guessing.

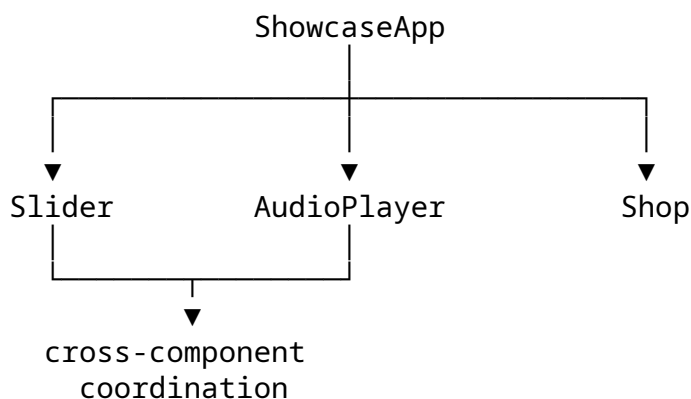


The implementation avoids silently trying unrelated selectors or continuing with missing required dependencies.

6. Application Composition

ShowcaseApp is the application composition root.

It receives the major components and owns their orchestration.



Its responsibilities are application-level:

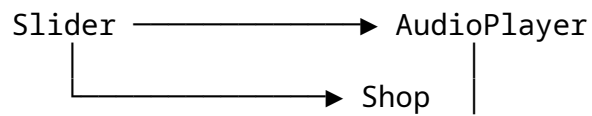
- initialize components;
- initialize managers;
- register event maps;
- route shared browser input;
- react to component events;
- keep Slider and AudioPlayer synchronized;
- coordinate visual modes.

It deliberately does **not** become the place where slider movement or audio playback itself is implemented.

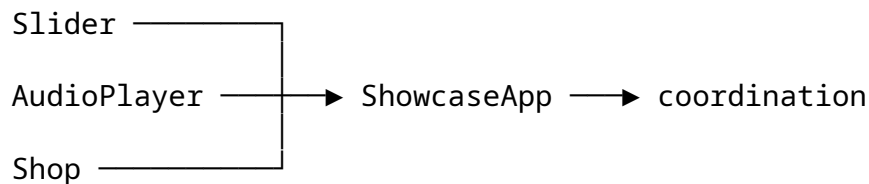
That boundary is important.

7. Components Do Not Directly Own Each Other

The undesirable dependency would look like:



Instead:



So:

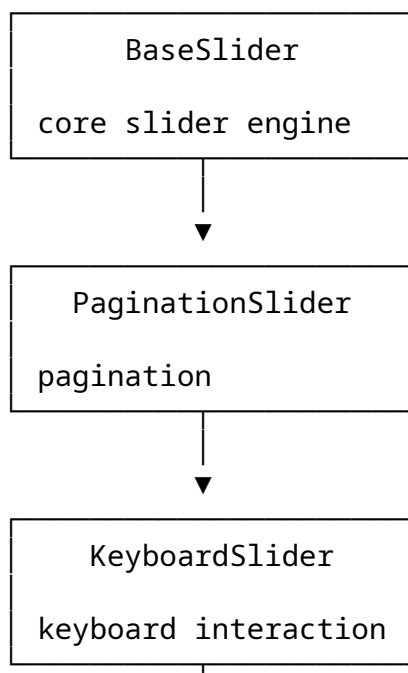
Slider does not need AudioPlayer internals
AudioPlayer does not need Slider internals
Shop does not need either implementation

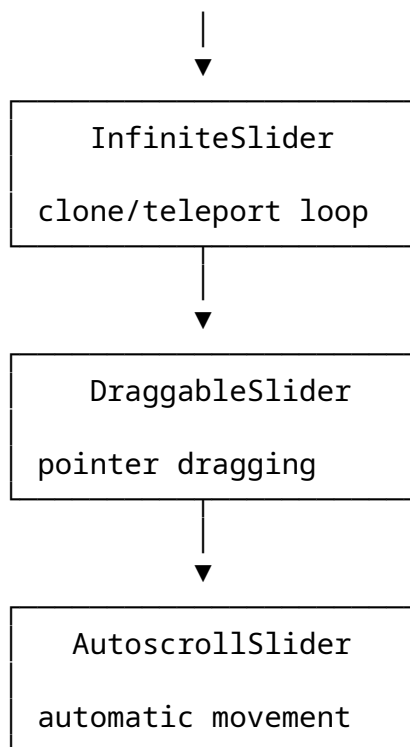
This is where ShowcaseApp can reasonably be described as a **Mediator-like application coordinator**.

It is not a second-level domain object that owns everything; it is the place where cross-component consequences are decided.

8. Slider Architecture

The slider is composed progressively through inheritance.





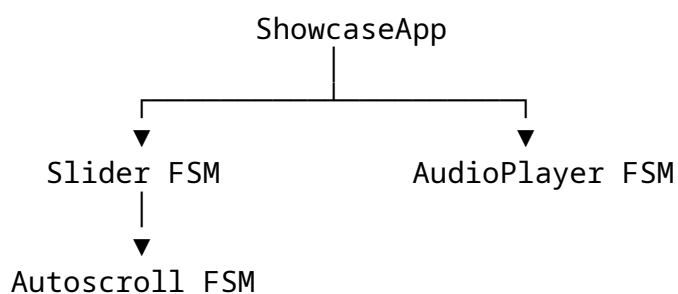
Each layer adds a capability or specializes an existing algorithm.

This makes the hierarchy an experiment in **polymorphic capability extension**, rather than a set of duplicated slider implementations.

9. Local State Machines

The application intentionally does not use one giant global state machine.

Instead, state belongs to the component that owns the corresponding behavior.

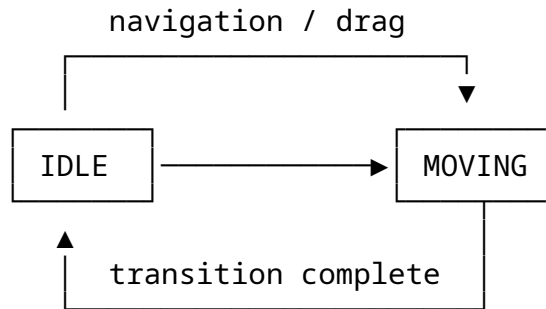


The project therefore contains three meaningful FSM domains:

1. Base slider movement
2. Autoscroll control
3. Audio playback mode

This keeps state ownership local and prevents ShowcaseApp from becoming coupled to internal states.

10. Slider FSM

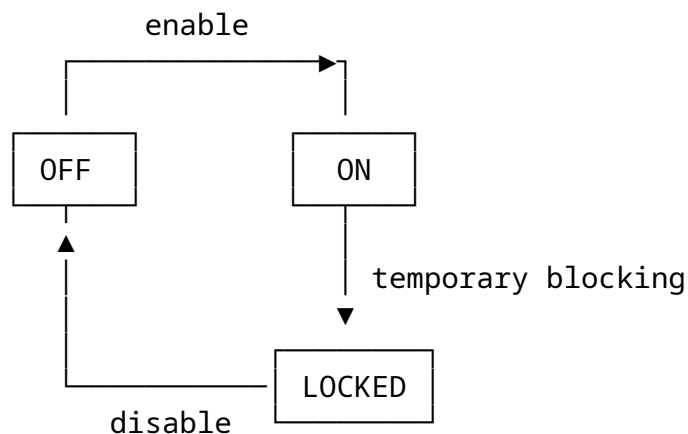


The important distinction is:

MOVING $\neq$ slidechange

A movement can begin before a slide becomes the committed active slide.
That distinction is used by application-level coordination.

11. Autoscroll FSM



LOCKED is conceptually different from ordinary OFF.

OFF = intentionally disabled

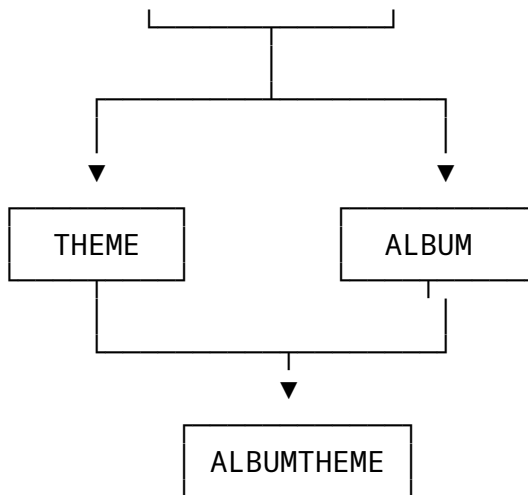
LOCKED = temporarily prevented from operating

That distinction becomes important when album playback takes control of the interaction.

12. Audio FSM

The AudioPlayer maintains its own playback-mode state.

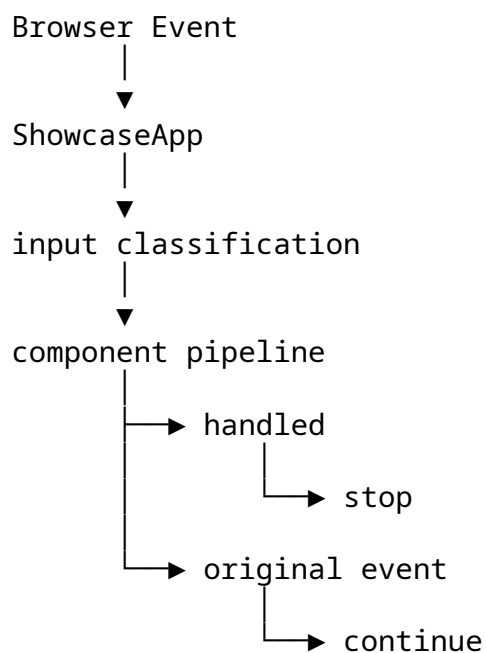




The exact transitions depend on the application action that caused the change; the important architectural point is that playback mode is state owned by AudioPlayer.

13. Input Pipeline

One of the defining mechanisms of the architecture is the `_pipe` model. A browser event is not necessarily owned by the first component that sees it.



The return value acts as a lightweight continuation protocol.

```

true
└─ event consumed

original Event
└─ event not consumed → pass it further
  
```

other result
└─ component/application-specific meaning

This is where the project most clearly resembles a **Chain of Responsibility**, combined with polymorphic handlers.

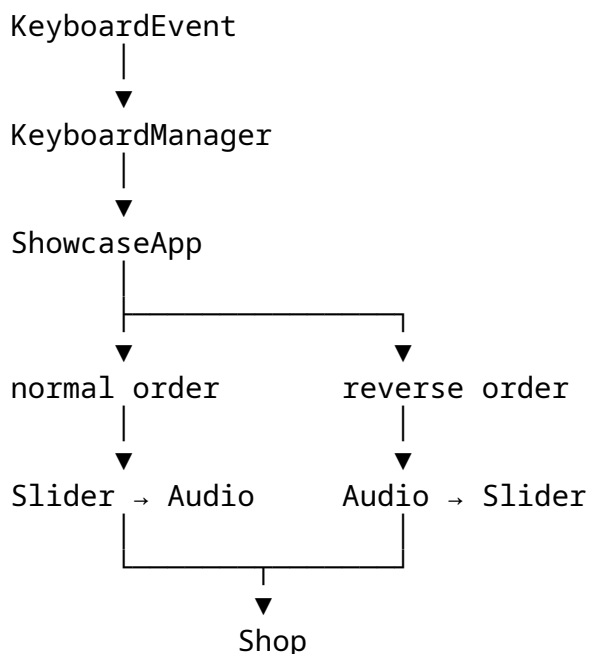
The implementation is intentionally lightweight rather than a textbook framework-level chain object.

14. Keyboard Pipeline

KeyboardManager classifies the key event.

ShowcaseApp decides the application-level routing mode.

The actual component handles the domain action.

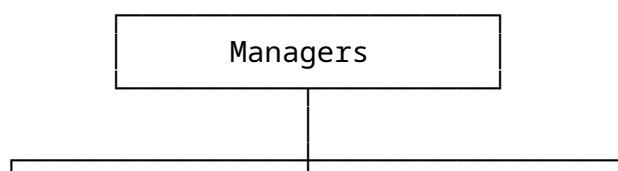


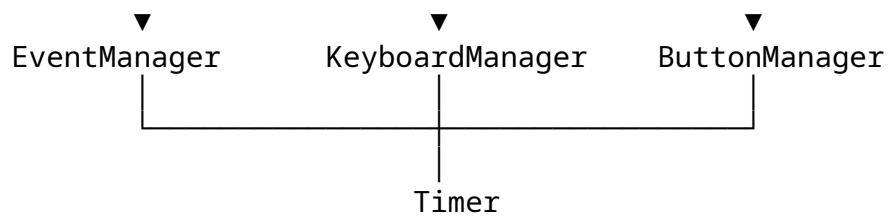
The reverse path matters when a keyboard interaction must give `AudioPlayer` the first opportunity to consume the input.

This is not merely a keyboard shortcut table; it is part of the application's input-routing design.

15. Managers

Managers isolate mechanisms that are reused by several components.





The principle is:

Manager = reusable interaction mechanism

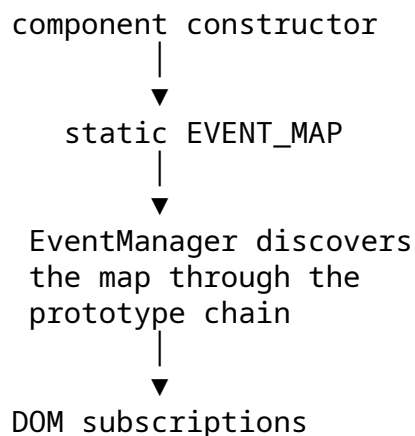
Component = meaning/domain behavior

This avoids putting generic DOM/event machinery directly into every component.

16. EventManager

EventManager supports two distinct mechanisms.

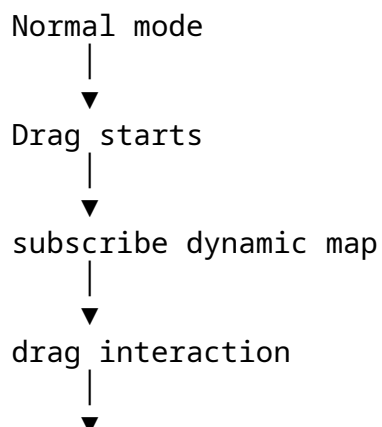
Declarative static maps



This allows inherited slider layers to contribute their event declarations.

Dynamic subscriptions

Some events exist only for a temporary interaction.



Drag ends



unsubscribe dynamic map

This is especially useful for pointer dragging.

17. KeyboardManager and ButtonManager

Both managers follow the same broad architecture.

configuration



lookup / normalization



handler contract validation



runtime action table



component method

The manager therefore knows **how to route input**, while the component knows **what the action means**.

This is close to a **Strategy** idea because behavior can be selected through configuration.

18. Button

The Button abstraction represents an executable UI action.

DOM interaction



Button



execute(event)

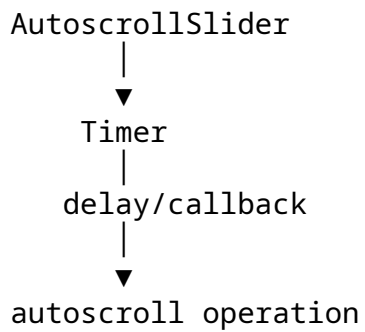


component command

It is best described as a **Command-like abstraction with a small DOM wrapper**, rather than forcing it into a textbook Command implementation.

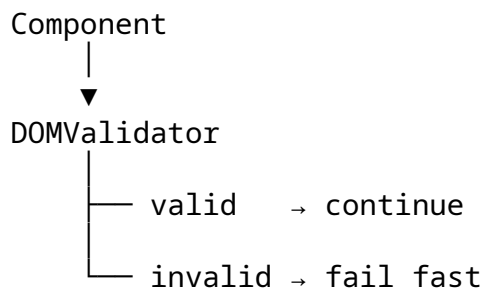
19. Other Infrastructure

Timer

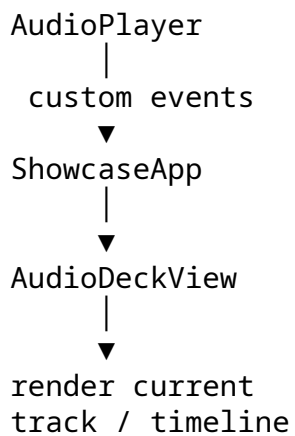


Timer knows about timing mechanics, not slider semantics.

DOMValidator



AudioDeckView



The view renders information; the AudioPlayer owns playback state.

20. Architectural Pattern Map

The project uses several recognizable patterns and principles, but they are not all textbook GoF implementations.

Pattern / Principle	Where	Purpose
Mediator-like coordination	ShowcaseApp	Coordinates independent components
Observer / Pub-Sub style	EventManager + custom events	Decouples event producers and consumers
Chain of Responsibility	_pipe + handler result protocol	Allows handlers to consume or forward input
Strategy-like configuration	KeyboardManager, ButtonManager	Selects actions from declarative configuration
Template Method-like inheritance	Slider hierarchy	Common algorithms with polymorphic extension points
Command-like abstraction	Button	Represents executable UI actions
FSM / state modeling	Slider, AutoscrollSlider, AudioPlayer	Localizes state and transitions
Composition	ShowcaseApp	Builds the application from independent parts
Dependency Injection	construction/init of components and managers	Supplies dependencies externally
Inversion of Control	application composition	Separates composition from component internals
Polymorphism	slider hierarchy/static contracts	Specializes behavior without duplication

A few important qualifications:

- Dependency Injection is an architectural technique, not a GoF pattern.
- IoC is a broader principle.
- FSM is a state-modeling technique; it does not automatically mean the State pattern.
- EventManager is better called Observer/Pub-Sub-like infrastructure than a textbook Observer implementation.
- Factory terminology should be used only where the implementation actually contains an appropriate creation abstraction.

21. Why There Is No Global State Pattern

A separate global state object would currently duplicate ownership.

The existing architecture is:

```
Slider
└─ slider FSM
```

```
AutoscrollSlider
└─ autoscroll FSM
```

```
AudioPlayer
└─ audio FSM
```

ShowcaseApp does not need to know every internal state.

Therefore:

local state ownership
>
global state registry

for the current scope.

A dedicated State-pattern implementation would make sense only if the application itself later develops genuine macro-level modes with state-specific behavior.

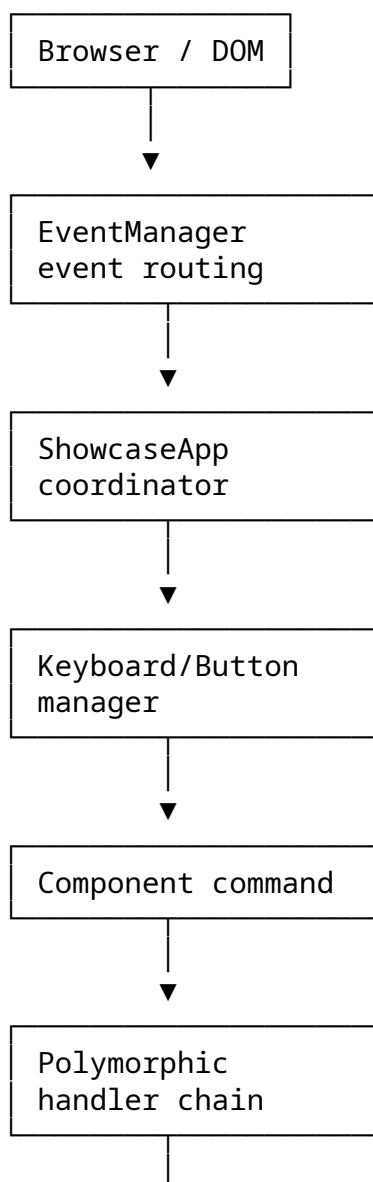
The principle is:

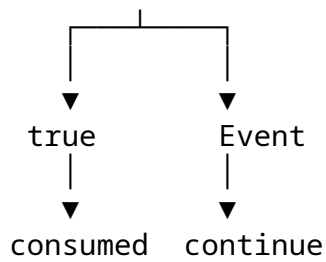
Introduce a pattern because a real problem requires it, not because the pattern exists.

22. Pattern Interaction

The most interesting architectural characteristic is not the number of patterns.

It is how they cooperate.





That is the architectural center of the project.

23. Architectural Philosophy

The project deliberately chooses:

complex but controllable
 over
 minimal but tightly coupled

The extra abstraction exists because the project is an architecture exercise.

The purpose is to investigate:

component boundaries
 +
 inheritance
 +
 polymorphism
 +
 FSM
 +
 managers
 +
 event routing
 +
 configuration
 +
 validation
 +
 application mediation

The architecture should therefore not be judged only by:

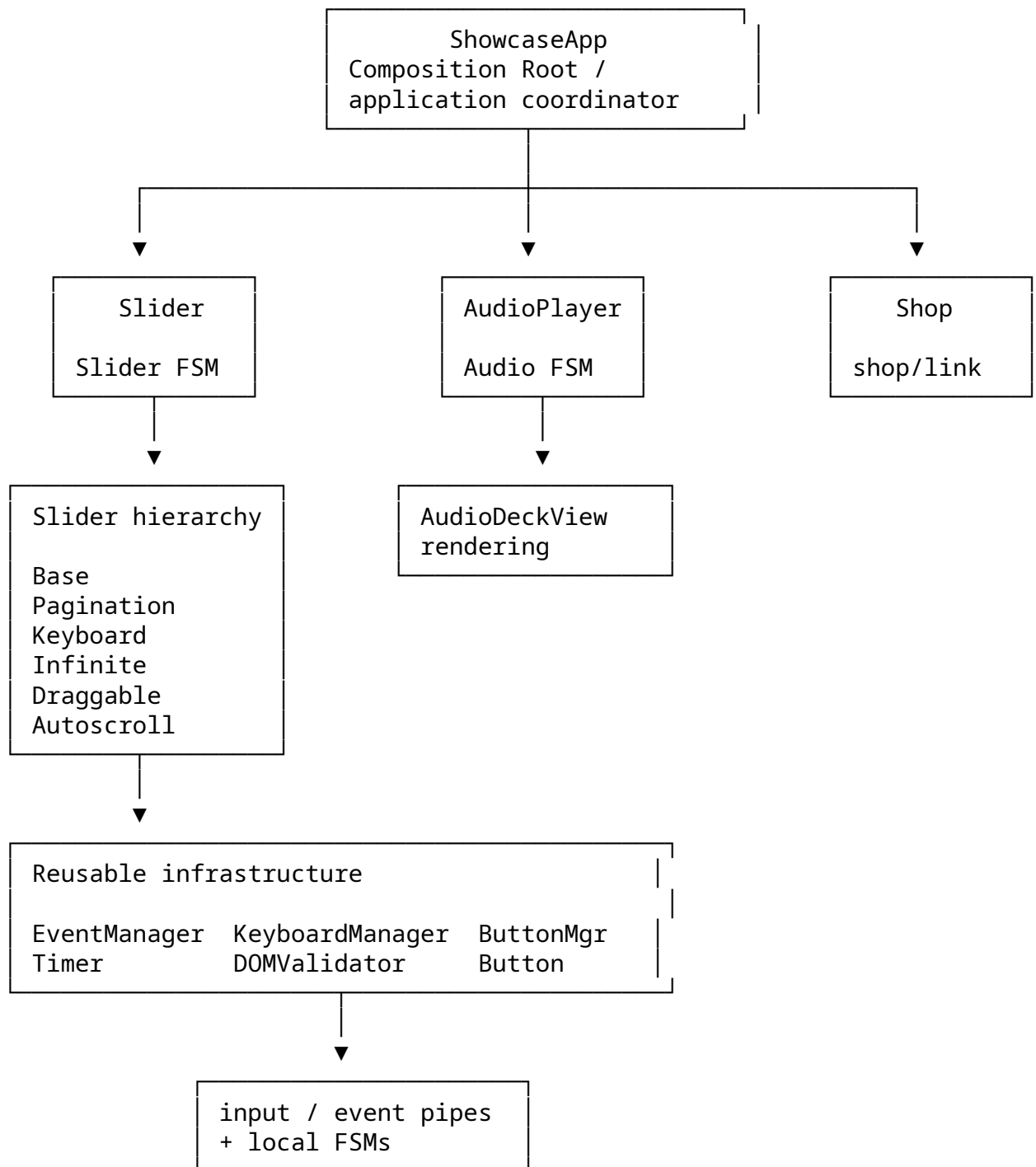
“Could the same slider be implemented in fewer lines?”

The more useful question is:

“Does the added structure create meaningful boundaries and reusable mechanisms?”

For this project, that is the intended trade-off.

24. Final Architecture



Summary

The architecture can be understood as five cooperating concerns:

1. Application orchestration
 - └ ShowcaseApp
2. Domain components
 - └ Slider
 - └ AudioPlayer
 - └ AudioDeckView

└─ Shop

3. Reusable interaction infrastructure

└─ EventManager
└─ KeyboardManager
└─ ButtonManager
└─ Timer

4. Validation / UI abstractions

└─ DOMValidator
└─ Button

5. Internal component architecture

└─ inheritance
└─ polymorphism
└─ FSMs
└─ input pipelines

The central architectural principle is:

Components own behavior and state

Managers own reusable interaction mechanisms

ShowcaseApp owns composition and cross-component coordination

Events and pipelines connect the layers

without exposing unnecessary implementation details